TEMCO LEARN

Temco Platform

Architecture · built Nov 10, 2026

Architecture

The repositories

TEXT

```
C:\QN\temcocontrols\
│
├─ T3000Webview-EEZ-Studio        the product repo — the content lives here
│   ├─ docs/t3000/**              the pages, one folder per chapter
│   ├─ docs/help/registry.json    the structure: areas, pages, owners, order
│   ├─ docs/help/index.json       generated: headings + git provenance
│   ├─ docs/toc.yml  **/toc.yml   generated navigation
│   ├─ docs/templates/t3000/      the site template: brand, page-meta bar
│   ├─ dist/docs/                 the rendered copy, and the package the site reads
│   └─ scripts/docs-*.mjs         the build side: registry, gates, changelog, media
│
├─ temco-platform                 this repository — the fork of VS Code: the shell, the
│   ├─ extensions/temco-docs/     the module — every documentation feature lives here
│   │   ├─ src/registry.ts        reads the registry, runs the structural checks
│   │   ├─ src/tree.ts            the Content view: areas, chapters, pages, gaps
│   │   ├─ src/diagnostics.ts     problems, in the file that has to be fixed
│   │   ├─ src/extension.ts       view, status bar, commands, file watcher
│   │   └─ src/…                  builds, report, scaffold, feedback, ai/ — the module t
│   │   └─ media/docs.svg         the view icon
│   ├─ docs/temco-learn/design/ the platform design (17 documents)
│   └─ product.json               branding of the application: name, icon, protocol, gal
│
└─ docfx                          the publishing engine — upstream, pinned
                                  never modified; a version bump only
```

The flow

TEXT

```
open a product repo in the editor
     │
     ▼
read     docs/help/registry.json   →  areas · chapters · pages · owners · order
     │
     ▼
show     Content view              →  pages, chapters, issues, never reviewed
     │                                Gaps → only the pages with a real fault
     ▼
check    file exists · id agrees with area/chapter · title unique · order unique
     │
     ▼
fetch    the runtime and a model are downloaded on request, into the user's own folde
     │
     ▼
serve    the local server starts on demand, on loopback only, and stops when it is id
     │
     ▼
draft    the local AI reads a commit range and proposes page edits
     │
     ▼
approve  a named human accepts; nothing else ever writes
     │
     ▼
publish  the engine         →  the package  →  the site
         the help build     →  CHM          →  ships with the installer
```

How it ships

```
TEXT

   the application   →  TemcoStudio-Setup.exe
                        ├─ the branded build of VS Code  ("Temco Platform")
                        └─ + the module, built in — one installer, one update path

   plain VS Code     →  the same module, run from the platform's sources
                        (it uses the published `vscode` API only — nothing shell-specifi

   same module · same views · same checks · same AI — the only difference is who builds

   what a customer installation gets

   T3000
     └─ T3000_Help.chm      the only artifact from this platform
                            no extension, no editor, no AI, no registry
```

What it publishes (target)

```
TEXT

   /                          the home page: what is inside, what changed recently
   /t3000/latest/…            a product, current — the pages exactly as they are today
   /t3000/13.0/…              the same pages for a pinned version
   /t3000/13.0/downloads      the PDF, the help file and the offline copy
   /_admin/                   health: owners, gaps, staleness — noindex
   /firmware/                 a second product later, from the same pipeline
```

The three boundaries that must not blur

1. **Content never moves.** Pages stay at `docs/t3000/**` in the product repo, which is why the application, the MCP tools and the F1 help keep working untouched. This extension reads them, and writes to one only when a maintainer asks and confirms the exact path: the file for a page the registry declares, a draft they applied, or an empty registry where a folder they have open has none. Nothing under `_admin/` or `_edit/` is ever written — those are generated pages — and nothing else in the folder is touched. Nothing is ever committed. One deliberate exception: the platform **design** lives at `docs/temco-learn/design/`, because the design of a thing belongs with the thing. The product repo keeps the content and the pipeline; it does not register or publish the design pages.

2. **The application is packaging, not features.** The shell holds the branding, the default layout and the build. A module must never depend on something only the shell provides, or it stops working in plain VS Code.
3. **One artifact crosses into a customer installation.** The CHM. No editor, no registry, no AI runtime.

Decisions already made, so they are not re-litigated

- The publishing engine is **pinned, never patched** unless a real need appears; the fork of it, if it ever happens, lives as commits in the project's own repo, not as loose patch files.
- One repository for the whole product. No submodules in either direction — the module is built in, not consumed as a package.
- The AI drafts and a named human approves. Nothing publishes itself.